UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CURSO: ORGANIZACIÓN DE LENGUAJES Y COMPILADORES
CATEDRÁTICO: ING. MOISES GRANADOS

PROYECTO 1: GENERADOR DE FORMULARIOS
MANUAL DE USUARIO

ESTUDIANTE	**CARNÉ:**
Cristian Alejandro Roldán López	202147280

QUETZALTENANGO, 24 DE MARZO 2,026

Requerimientos:
Android versiones: 7 en adelante
Almacenamiento: 25mb
Ram: 4GB

Descripción de la actividad

En la actualidad los dispositivos móviles son herramientas necesarias para el desarrollo de las actividades diarias, es importante conocer el proceso de desarrollo de aplicaciones hechas para esa plataforma.

Asimismo, una aplicación que facilite la creación de formularios sería de gran utilidad, ya que permitiría, desde nuestro teléfono, generar una gran variedad de formularios personalizados. Por ello, se propone el desarrollo de una aplicación que permita construir formularios a partir de comandos ingresados por el usuario.

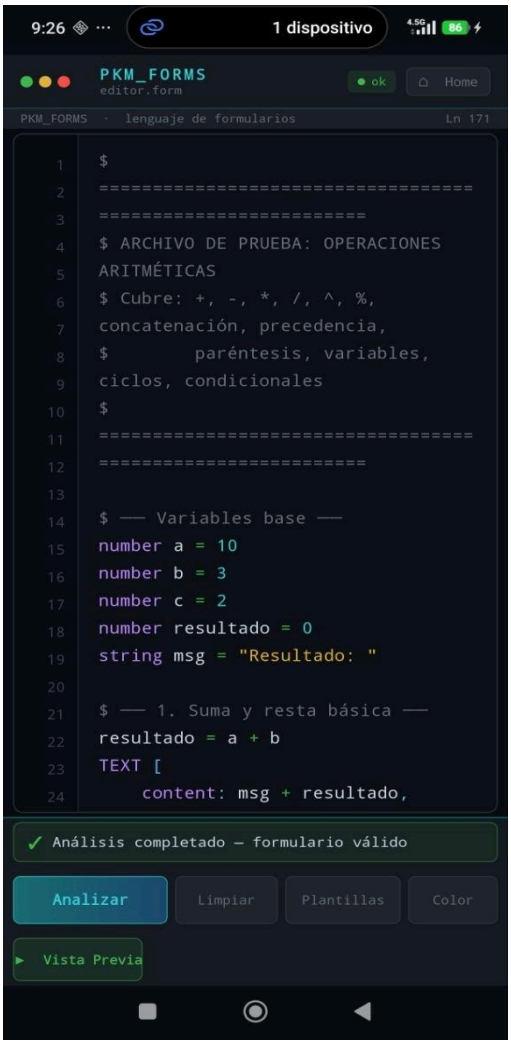
Pantalla de inicio

Crear / Editar: Apartado para poder escribir el código que se desea analizar.

Responder formularios: Apartado donde se encuentran los formularios para poder responderlos.

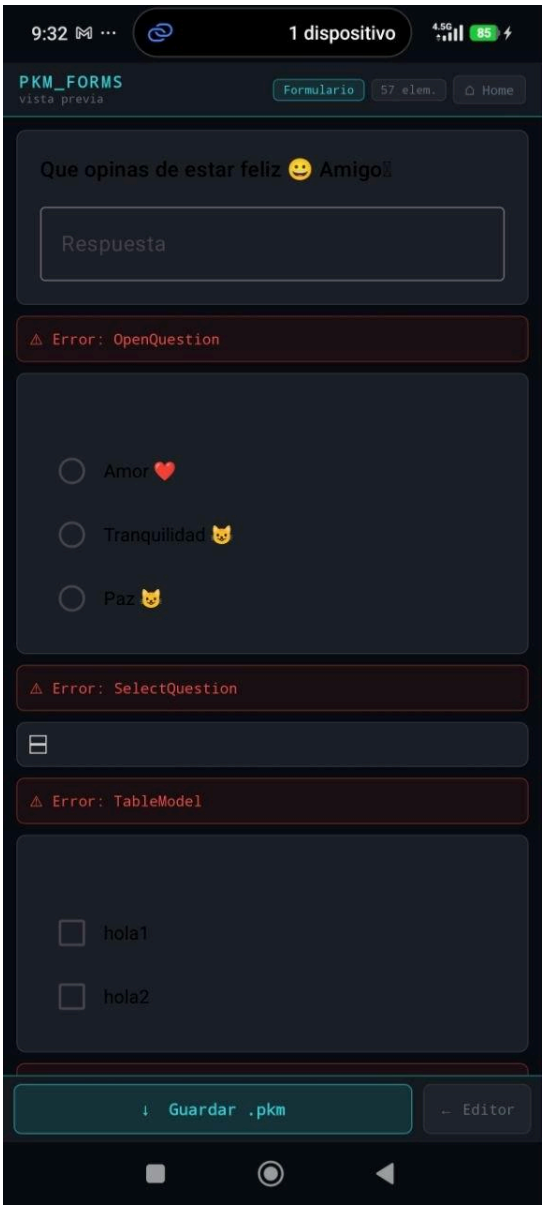
Servidor: Apartado para poder descargar y subir archivos a un servidor.

Forma de generar un formulario



Analizar: Presionando se analiza el código ingresado para generar un formulario.
Limpiar: Limpia el apartado para agregar código.
Plantillas: Apartado con plantillas predefinidas que se pueden editar.
Color: Nos sirve para poder agregar colores en los distintos formatos si no sabemos el código.
Vista previa: Muestra la previsualización de como quedará el formulario.

Vista previa.

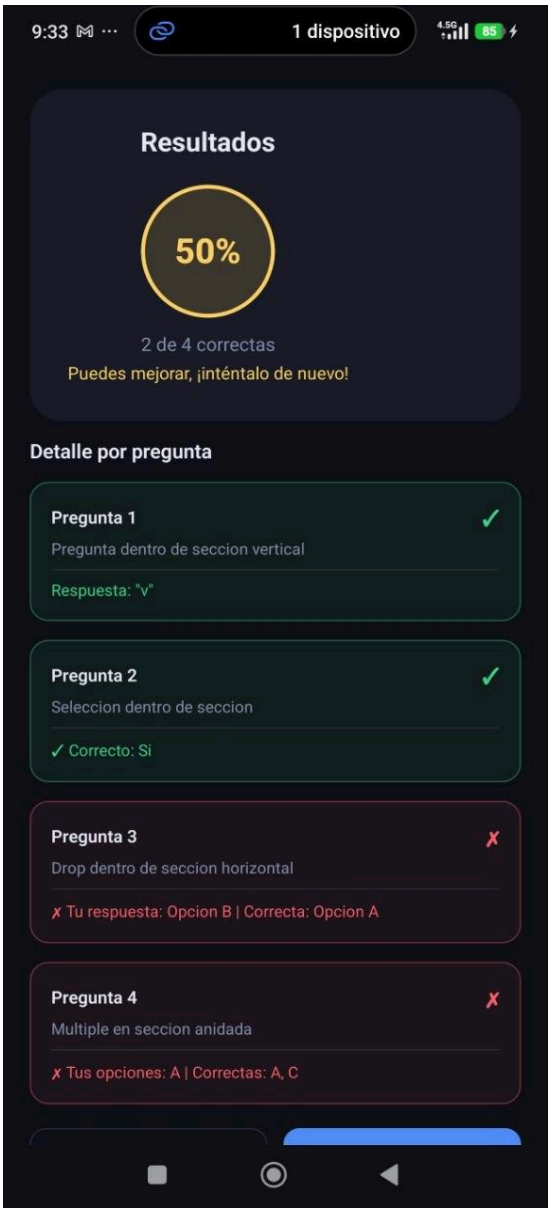


Guardar .pkm: Funciona para guardar el formulario creado en el formato .pkm.

Formularios guardados.

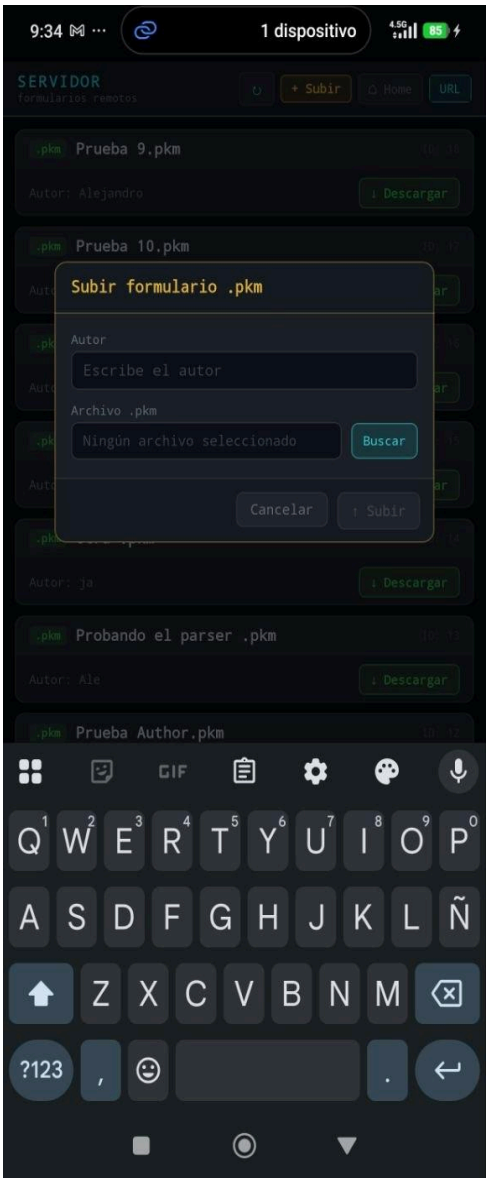
Responder: Funciona para responder el formulario creado.
Eliminar: Se elimina el formulario que no se desea.

Forma de responder formularios y visualizar los aciertos.

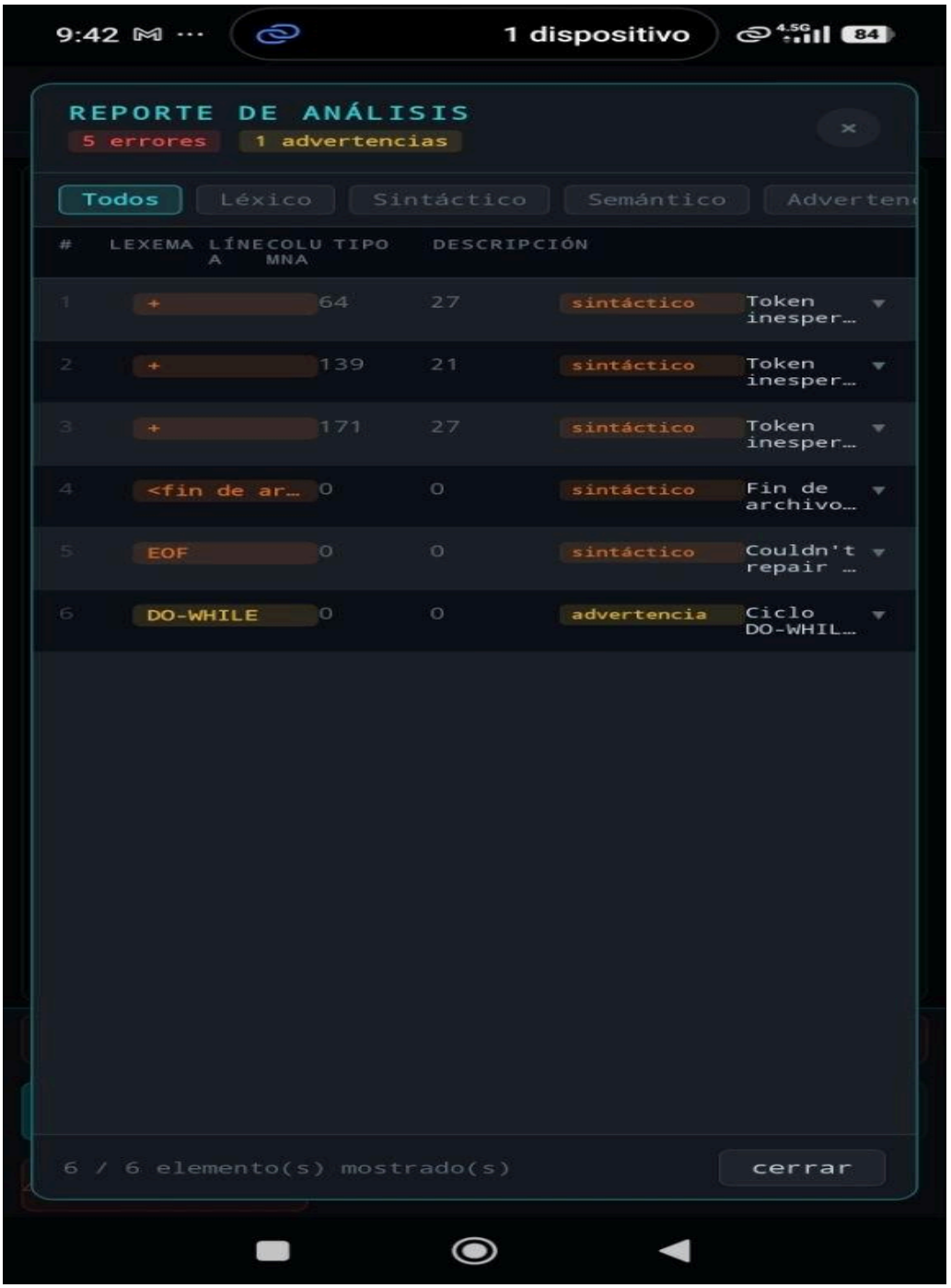


Servidor

9:34
1 dispositivo
SERVIDOR
formularios remotos
+ Subir
Home
URL
.pkm Prueba 9.pkm
Autor: Alejandro
↓ Descargar
.pkm Prueba 10.pkm
Autor: Cristian
↓ Descargar
.pkm JA.pkm
Autor: hhjhjh
↓ Descargar
.pkm JAJAJA.pkm
Autor: dfds
↓ Descargar
.pkm Otra .pkm
Autor: ja
↓ Descargar
.pkm Probando el parser .pkm
Autor: Ale
↓ Descargar
.pkm Prueba Author.pkm
Autor: Probando
↓ Descargar
.pkm Otra prueba .pkm
Autor: a
↓ Descargar
.pkm Prueba .pkm
Autor: g
↓ Descargar
.pkm OjalÃ¡ lo guarde.pkm
Autor: Pruebas
↓ Descargar


9:34
1 dispositivo
SERVIDOR
formularios remotos
+ Subir
Home
URL
Prueba 9.pkm
Autor: Alejandro
↓ Descargar
Prueba 10.pkm
Subir formulario .pkm
Autor
Escribe el autor
Archivo .pkm
Ningún archivo seleccionado
Buscar
Cancelar
↑ Subir
Autor: ja
↓ Descargar
Probando el parser .pkm
Autor: Ale
↓ Descargar
Prueba Author.pkm
GIF
Q W E R T Y U I O P
A S D F G H J K L Ñ
Z X C V B N M
?123

Reportes


9:42
1 dispositivo
REPORTE DE ANÁLISIS
5 errores
1 advertencias
Todos
Léxico
Sintáctico
Semántico
Adverten
LEXEMA LÍNEA COLUMNA TIPO DESCRIPCIÓN
1 + 64 27 sintáctico Token inesper…
2 + 139 21 sintáctico Token inesper…
3 + 171 27 sintáctico Token inesper…
4 <fin de ar… 0 0 sintáctico Fin de archivo…
5 EOF 0 0 sintáctico Couldn't repair …
6 DO-WHILE 0 0 advertencia Ciclo DO-WHIL…
6 / 6 elemento(s) mostrado(s)
cerrar

Operadores aritméticos

Dentro del lenguaje se podrán utilizar, valores numéricos con o sin punto decimal con los cuales se pueden ejecutar operaciones aritméticas básicas las cuales se definen en la siguiente tabla.

Nota: Hay que tener en cuenta la operación unaria, es decir números con signos.

Símbolo	Descripción	Precedencia (de menor a mayor)
+	Suma	1
-	Resta	1
*****	Multiplicación	2
/	División	2
^	Potenciación	3
%	Módulo	3
( )	Paréntesis	4

Operadores de comparación

Se usan para comparar diferentes valores

Símbolo	Descripción	Precedencia (de menor a mayor)
>	Mayor que	0
>=	Mayor o igual que	0
<	Menor que	0
<=	Menor o igual que	0
==	Igualdad	0

Number y String

Ejemplo de uso:

```
/*
    Se puede crear la variable y se le deberá asignar un valor por defecto
    para los number es 0
    y para los strings una cadena vacía

*/
number myWidth
string name

/*
    También se puede inicializar de una vez la variable con un valor
*/
number myHeight = 10.0
string lastname = "Hola"

/*
    Posteriormente se le puede cambiar de valor,
    pero hay que tener en cuenta que redefinir la variable no es posible
*/
name = "pokemon"
```

```
/*
    Puede ser cualquiera de las preguntas
    que se especifican en la sección correspondiente

    Este tipo de variable si es necesario inicializarla siempre
*/
special myQuestion = OPEN_QUESTION [
    width: number,
    height: number,
    label: number
]

/*
    La variable especial se usa de la siguiente manera
    La invocación de la función por defecto de esta esta
    sirve para agregar lo que contiene al layout
    en este caso la pregunta abierta
*/
myQuestion.draw()
```

Dentro de la sintaxis de una variable special se pueden definir comodines que servirán para poder tener preguntas dinámicas, ejemplo:

```
/*
    Comodín = ?
    Un comodín puede existir dentro de una expresión
    Así como venir suelto
*/
special myQuestion = OPEN_QUESTION [
    width: ?,
    height: 10 + ?,
    label: number,
]
```

```
SECTION [
      with: number, $ ancho de toda la sección
      height: number, $ alto de toda la sección

      $ referencian a la esquina superior izquierda donde empieza la
sección
      pointX: number,
      pointY: number,

      /*
```

```
        Cómo se irán agregando los elementos, esto es opcional
        Si no se especifica se asume que es VERTICAL

        VERTICAL: De arriba hacia abajo
        HORIZONTAL: De izquierda a derecha
    */
    orientation: VERTICAL,

    $ La sección de elementos es opcional
    elements: {
          $ elementos aquí, separados por comas
          $ no debe haber una coma en el último elemento
    },

    /*
        Los estilos son opcionales
        y todo lo que se especifica dentro de los corchetes también
        aunque debe venir al menos un estilo, es decir no puede venir
vacio
        Estos estilos se heredan a los elementos dentro de la sección,
        salvo que los elementos tengan estilos personalizados
    */
    styles [
        "color": color,
        "background color": color, $ color del texto
        "font family": MONO, $ opciones posibles: MONO,
SANS_SERIF,CURSIVE
        "text size": number,
        /*
            El número representa el grosor del borde
            tipos posibles: LINE, DOTTED, DOUBLE
        */
        "border": (number, TYPE, color)
    ]
]
```

Las secciones se pueden anidar. Ejemplo:

```
SECTION [
    width: number,
    height: number,
    pointX: number,
    pointY: number,

    elements: {
```

```
            SECTION [ ... ],
            SECTION [ ... ]
      }
]
```

ablas

Se pueden definir tablas de la siguiente manera:

```
TABLE [
    width: number,
    height: number,
    pointX: number,
    pointY: number,

    elements: {
        [ $ primera línea
            {
                $ elemento 1
            },
            {
                $ elemento 2
            },
            {
                $ elemento 3
            }
        ],
        [ $ segunda línea
            {
                $ elemento 4
            },
            {
                $ elemento 5
            },
            {
                $ elemento 6
            }
        ]
    },

    styles [
          "color": color,
          "background color": color,
```

```
            "font family": MONO,
            "text size": number,
            "border": (number, TYPE, color)
    ]
]
```

```
TEXT [
    width: number,  $ opcional
    height: number, $ opcional
    content: "string",

    $ opcional
    styles [
         "color": color
        "background color": color
        "font family": MONO,
        "text size": number
    ]
]
```

Preguntas

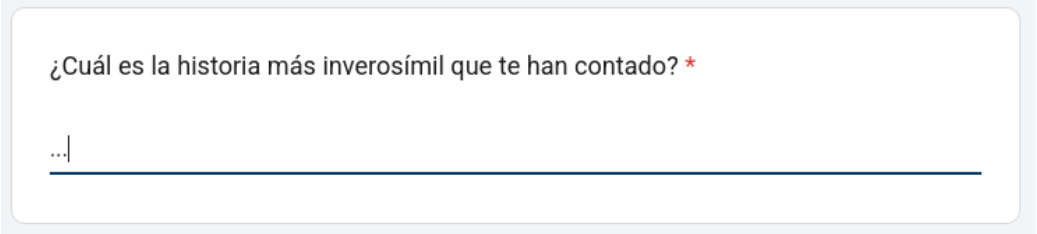
Todas las preguntas tendrán que ir dentro de una sección o una tabla necesariamente, así como el texto, estas heredan ciertos atributos de sus padres, como los estilos el alto y el ancho, aunque se pueden agregar.

Preguntas abiertas

Es una pregunta donde el usuario puede escribir una respuesta. La sintaxis para crearla es la siguiente:

```
OPEN_QUESTION [
    width: number, $ opcional
    height: number, $ opcional
    label: "string",

    $ los estilos son opcionales
    styles [
            "color": color,
            "background color": color,
            "font family": MONO,
```

```
            "text size": number
    ]
]
```



Preguntas desplegables

Son aquellas con opciones desplegables.

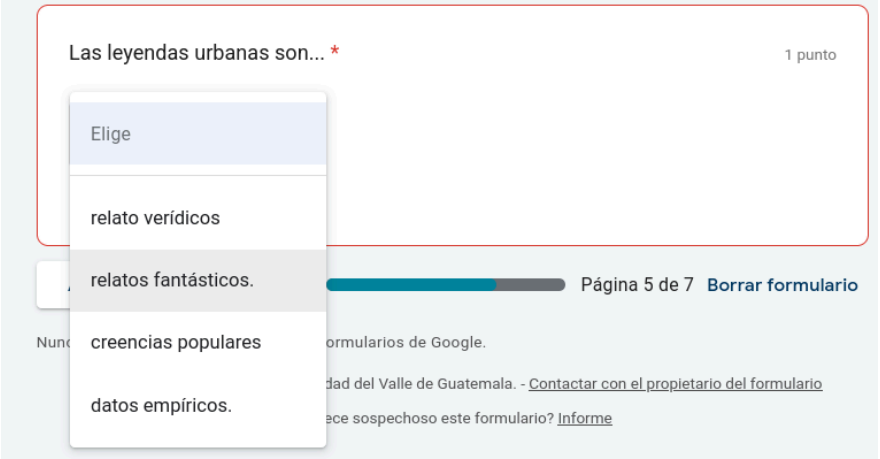
```
DROP_QUESTION [
    width: number, $ opcional
    height: number, $ opcional
    label: "string",

    options: {"first", "second"},

    /*
       La siguiente especificación es opcional
       Se utiliza para especificar el índice de la respuesta correcta
     se debe mostrar un error si el número se sale del rango de opciones
posibles
     */
    correct: number,

    $ los estilos son opcionales
    styles [
          "color": color,
          "background color": color,
          "font family": MONO,
          "text size": number
    ]
]
```

Debido a que muchas veces a los usuarios no se les ocurre ninguna opción, en lugar de especificar las respuestas, se puede ir a extraer un rango de pokemones según su número en la pokedex, desde el rango n hasta el m usando la siguiente función especial:

```
$buscar pokemons según el rango del número en la pokédex
who_is_that_pokemon(NUMBER, 1, 10)
```

Esto se logrará haciendo un REQUEST a la PokéAPI.
Nota: Al guardar las respuestas en el archivo pkm se debe guardar como si el usuario lo hubiera escrito.

Ejemplo de visualización de la pregunta:

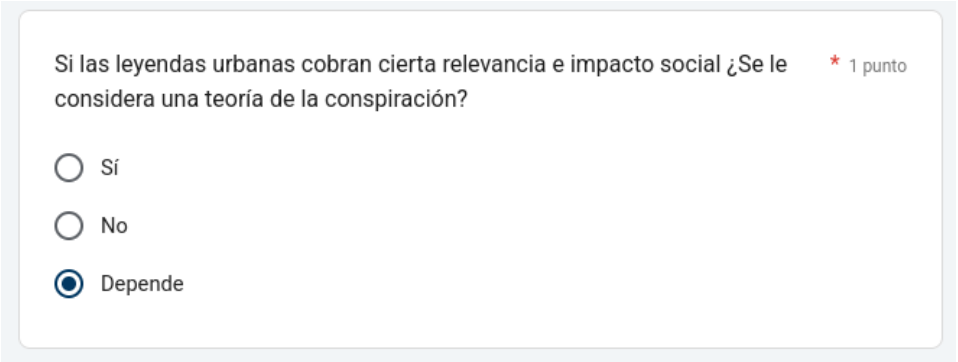


Preguntas de selección única

Es una pregunta donde el usuario puede seleccionar solo una respuesta. La sintaxis para crearla es la siguiente:

```
SELECT_QUESTION [
    width: number, $ opcional
    height: number, $ opcional
    options: {"first", "second", "third"},
    label: "string",

    $ opcional. Se debe mostrar un error si se sale del rango
    correct: number,

   $ los estilos son opcionales
    styles [
          "color": color,
          "background color": color,
          "font family": MONO,
          "text size": number
    ]
]
```

Si el usuario ingresa más de 5 opciones, se le debe mostrar una advertencia antes de agregar la pregunta al formulario actual.



Preguntas de selección múltiple

Es una pregunta donde el usuario puede seleccionar varias respuestas. La sintaxis para crearla es la siguiente:

```
MULTIPLE_QUESTION [
    width: number, $ opcional
    height: number, $ opcional
    options: {"first", "second", "third"},

    $ opcional
    correct: {number, number},

    $ los estilos son opcionales
    styles [
          "color": color,
          "background color": color,
          "font family": MONO,
          "text size": number
   ]
]
```

Bloques de código

Son bloques de código que incluyen lógica. Todos los bloques de código se pueden anidar.

Condicional

Se puede usar un condicional para mostrar algunas preguntas solo si se cumple una condición, la sintaxis es la siguiente:

```
IF ( 10 > 10 ) {
    /*
```

```
        Definición o asignación de variables,
        Preguntas
        U otros bloques de código
    */
} ELSE IF ( 10 < 50 ) {
      $ Esta sección es opcional
} ELSE  {
      $ Esta sección es opcional
}
```

Es importante saber que en este caso para que se cumpla la condición la expresión que se evaluará tiene que ser mayor o igual a 1, no se trabajarán con booleanos de 0 y 1, el valor puede ser mayor. Si es 0 o menor el condicional es falso. Esto aplica para todos los condicionales.

Ciclos while y do-while

```
WHILE ( 10 > 10 ) {
      $ instrucciones
}
```

```
DO {
   $ instrucciones
} WHILE ( 10 < 10 || 10 < 10 )
```

Ciclo For

En la parte de la instrucción en el for se asume que se declara la variable (si aún no existe) y se asume también el tipo number.
Si dicha variable ya se ha declarado antes y no es de tipo number, se debe mostrar un error.

```
FOR (i = 0 ; i <= 10 ; i = i + 1 ) {

}
```

Otra forma de usar un ciclo for es la que se muestra a continuación, donde se asume que la variable tiene que ir desde n hasta m. Las reglas de definición y validación de tipos de la variable utilizada aquí son las mismas

```
FOR (i in 1 .. 5) {

}
```

DISFRUTA DE LA APLICACIÓN ME LLEVO MUCHOS DESVELOS Y ESTRÉS A CADA MOMENTO.
ROLDÁN7